

28-rest-api-design

28. REST API Design

Level: □ Intermediate

Jam: 8 (4 minggu × 2 sesi)

Prasyarat: JavaScript/TypeScript dasar, Express.js

Output: REST API documented with OpenAPI + production-ready

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Merancang REST API dengan prinsip-prinsip REST (stateless, resource-based, uniform interface)
- Membuat URL design yang konsisten dan mengikuti best practices
- Mendokumentasikan API secara otomatis dengan OpenAPI 3.0 + Swagger UI
- Menangani error secara terstruktur (Problem Details RFC 7807)
- Menerapkan pagination, sorting, filtering, dan rate limiting
- Mengelola versioning API dengan strategi backward compatibility
- Mengamankan API dengan API keys, JWT, CORS, helmet, dan logging

Materi

Sesi	Topik	File
1	REST Principles — constraints, URL naming, HTTP methods & status codes	01-rest-principles.md
2	OpenAPI & Swagger — spec structure, paths, schemas, swagger-jsdoc setup	02-openapi-swagger.md

Sesi	Topik	File
3	Error Handling & Pagination — RFC 7807, offset vs cursor, sorting/filtering	03-error-pagination.md
4	Versioning & Security — URL/header/query versioning, JWT, CORS, helmet, logging	04-versioning-security.md

Output Akhir Modul

Production-ready REST API — Express/TypeScript API dengan dokumentasi OpenAPI otomatis, error handling terstruktur, pagination, versioning, security (JWT + CORS + helmet), dan logging.

AI Prompt Exercises

Sepanjang modul, latihan pake AI: - “Generate OpenAPI 3.0 spec for this endpoint” - “Review my API design — what REST principles am I violating?” - “Help me choose: offset pagination vs cursor pagination for this use case” - “Generate JWT middleware for Express with TypeScript” - “Write error responses following RFC 7807 format” - “What security headers should I add to this Express app?”

28.1 REST Principles

Apa itu REST?

REST (Representational State Transfer) — gaya arsitektur API yang memanfaatkan protokol HTTP. Bukan framework/library, tapi **pola desain**.

6 REST Constraints

Constraint	Penjelasan
Stateless	Setiap request berdiri sendiri. Server gak nyimpen session/client context

Constraint	Penjelasan
Client-Server	Pemisahan client dan server biar masing-masing bisa develop independent
Cacheable	Response harus explicitly menyebut cacheable atau gak via Cache-Control
Uniform Interface	Interface konsisten: resource identification, representation manipulation, self-descriptive messages, HATEOAS
Layered System	Client gak perlu tau apakah dia ngomong langsung ke server atau lewat proxy/load balancer
Code on Demand (optional)	Server bisa kirim executable code (JS) ke client

Stateless — Contoh

```
// ❌ BAD – nyimpen state di server
const sessions = new Map<string, { userId: number }>();

app.get('/api/profile', (req, res) => {
  const token = req.headers['authorization'];
  const session = sessions.get(token);
  if (!session) return res.status(401).json({ error: 'Unauthorized' });
  // ...
});

// ✅ GOOD – semua state dikirim client tiap request (JWT)
app.get('/api/profile', (req, res) => {
  const token = req.headers['authorization'];
  const payload = verifyJwt(token); // stateless – tinggal verify
  // ...
});
```

Resource-Based — URL Naming

```
// ❌ BAD – pake verb di URL
GET /api/getUsers
POST /api/createUser
GET /api/deleteUser?id=5

// ✅ GOOD – pake nouns (resources)
```

```

GET    /api/users           // list users
POST   /api/users           // create user
DELETE /api/users/:id      // hapus user
GET    /api/users/:id    // detail user

```

HTTP Methods & Status Codes

Methods

Method	Fungsi	Idempotent	Safe
GET	Ambil resource	☐	☐
POST	Create resource	☐	☐
PUT	Replace resource (full update)	☐	☐
PATCH	Partial update	☐*	☐
DELETE	Hapus resource	☐	☐

*PATCH gak idempotent secara spec kalo pake JSON Patch (RFC 6902), tapi banyak implementasi bikin idempotent. Hati-hati.

Status Codes — Lengkap

Kode	Nama	Kapan Pake
200	OK	Sukses GET, PUT, PATCH
201	Created	Sukses POST (create)
204	No Content	Sukses DELETE (gak ada response body)
301	Moved Permanently	Resource pindah URL
304	Not Modified	Cache masih valid (ETag/If-None-Match)
400	Bad Request	Input gak valid
401	Unauthorized	Belom login / token gak valid
403	Forbidden	Udah login tapi gak punya akses
404	Not Found	Resource gak ditemuin
405	Method Not Allowed	Endpoint ada tapi method salah
409	Conflict	Duplicate / state conflict
422	Unprocessable Entity	Validasi logic (contoh: email udah terdaftar)
429	Too Many Requests	Rate limit exceeded

Kode	Nama	Kapan Pake
500	Internal Server Error	Error server (jangan bocorin detail)
502	Bad Gateway	Upstream server error
503	Service Unavailable	Maintenance / overload

```
// Contoh implementasi
import express, { Request, Response } from 'express';
const app = express();
app.use(express.json());

interface User {
  id: number;
  name: string;
  email: string;
}

const users: User[] = [];

// GET /api/users - 200
app.get('/api/users', (_req: Request, res: Response) => {
  res.status(200).json({
    success: true,
    data: users,
    total: users.length,
  });
});

// POST /api/users - 201
app.post('/api/users', (req: Request, res: Response) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ // 400 - Bad Request
      success: false,
      message: 'Name dan email wajib diisi',
    });
  }
  const newUser: User = { id: users.length + 1, name, email };
  users.push(newUser);
  res.status(201).json({ success: true, data: newUser });
});

// DELETE /api/users/:id - 204
app.delete('/api/users/:id', (req: Request, res: Response) => {
```

```

const id = parseInt(req.params.id);
const index = users.findIndex(u => u.id === id);
if (index === -1) {
  return res.status(404).json({ // 404 – Not Found
    success: false,
    message: 'User gak ditemuin',
  });
}
users.splice(index, 1);
res.status(204).send(); // 204 – No Content
});

```

URL Design Best Practices

1. Plural nouns

- /api/users
- /api/products
- /api/user
- /api/product

2. Hierarchical resources

```

GET /api/users/:id/orders // orders milik user tertentu
GET /api/users/:id/orders/:orderId // detail order
POST /api/users/:id/orders // bikin order baru

```

3. Query params untuk filter

```

GET /api/users?role=admin&status=active
GET /api/products?category=electronics&minPrice=100000
GET /api/orders?startDate=2024-01-01&endDate=2024-12-31

```

4. Versioning

- /api/v1/users
- /api/v2/users

(Lebih detail di sesi 4 — Versioning & Security)

5. Consistent casing — pake kebab-case atau camelCase

- /api/users/:userId/order-items
- /api/users/:userId/orderItems
- /api/users/:user_id/order_items (snake_case di URL jarang dipake)

Tapi **camelCase** lebih umum di JSON response.

Latihan

1. Refactor URL berikut jadi RESTful: GET /api/getAllProducts, POST /api/createProduct, GET /api/deleteProduct?id=5. Tulis Express route handler versi RESTful-nya.
2. Buat endpoint GET /api/users/:id/orders yang return orders milik user tertentu. Handle status code 200 (ketemu), 404 (user gak ada), 400 (id bukan angka valid). Tulis kode TypeScript Express.
3. Dari daftar status code: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500. Buat Express middleware/helper function sendSuccess(res, data, statusCode?) dan sendError(res, message, statusCode?) yang konsisten.
4. Bikin route PATCH /api/users/:id buat partial update user. Validasi: kalo body kosong return 400. Kalo user gak ditemui return 404. Kalo email udah dipake user lain return 409. Pake kode TypeScript Express.

28.2 OpenAPI & Swagger

Kenapa Dokumentasi API Penting?

- Frontend/dev lain tau endpoint apa aja yang tersedia
- Bisa auto-generate client library (OpenAPI Generator)
- Testing jadi lebih gampang (Swagger UI punya "Try it out")
- Contract antara backend & frontend

OpenAPI 3.0 Structure

OpenAPI spec file (YAML atau JSON) punya struktur:

```
openapi: "3.0.3"
info:
  title: "Todo API"
  description: "API manajemen todo sederhana"
  version: "1.0.0"
  contact:
    name: "Tim RPL"
    email: "rpl@sekolah.id"
servers:
  - url: "http://localhost:3000/api/v1"
```

```

    description: "Development server"
  paths:
    /todos:
      get:
        summary: "Ambil semua todos"
        responses:
          "200":
            description: "List todos"
            content:
              application/json:
                schema:
                  $ref: "#/components/schemas/ToDoList"
  components:
    schemas:
      Todo:
        type: object
        properties:
          id:
            type: integer
          title:
            type: string
          completed:
            type: boolean
      ToDoList:
        type: array
        items:
          $ref: "#/components/schemas/Todo"

```

Komponen Utama OpenAPI

Bagian	Fungsi
openapi	Version spec (3.0.0 / 3.0.3 / 3.1.0)
info	Metadata API — title, version, description
servers	Base URL server (bisa multiple: dev/staging/prod)
paths	Definisi endpoint & operasi
components/schemas	Reusable data models (DRY)
parameters	Reusable parameters (query, path, header)
security	Security scheme (API key, JWT, etc)

Setup swagger-jsdoc + swagger-ui-express di Express

Install:

```
npm install swagger-jsdoc swagger-ui-express
npm install -D @types/swagger-jsdoc @types/swagger-ui-express
```

1. Define OpenAPI spec dengan JSDoc comments

```
// src/docs/swagger.ts
import swaggerJsdoc from 'swagger-jsdoc';

const options: swaggerJsdoc.Options = {
  definition: {
    openapi: '3.0.3',
    info: {
      title: 'Todo API',
      version: '1.0.0',
      description: 'API manajemen todo sederhana',
      contact: {
        name: 'Tim RPL',
        email: 'rpl@sekolah.id',
      },
    },
  },
  servers: [
    { url: 'http://localhost:3000/api/v1', description: 'Development' },
  ],
  components: {
    securitySchemes: {
      bearerAuth: {
        type: 'http',
        scheme: 'bearer',
        bearerFormat: 'JWT',
      },
    },
  },
  security: [{ bearerAuth: [] }],
  apis: ['./src/routes/*.ts'], // scan file routes buat JSDoc
};

export const swaggerSpec = swaggerJsdoc(options);
```

2. JSDoc annotations di route files

```
// src/routes/todos.ts
import { Router, Request, Response } from 'express';

const router = Router();
```

```

interface Todo {
  id: number;
  title: string;
  completed: boolean;
}

let todos: Todo[] = [
  { id: 1, title: 'Belajar REST API', completed: false },
];

/**
 * @openapi
 * /todos:
 *   get:
 *     tags: [Todos]
 *     summary: Ambil semua todos
 *     parameters:
 *       - in: query
 *         name: completed
 *         schema:
 *           type: boolean
 *         description: Filter berdasarkan status completed
 *     responses:
 *       200:
 *         description: List todos
 *         content:
 *           application/json:
 *             schema:
 *               type: array
 *               items:
 *                 $ref: '#/components/schemas/Todo'
 */
router.get('/todos', (req: Request, res: Response) => {
  let result = todos;
  if (req.query.completed !== undefined) {
    const completed = req.query.completed === 'true';
    result = todos.filter(t => t.completed === completed);
  }
  res.json(result);
});

/**
 * @openapi
 * /todos:
 *   post:

```

```

*      tags: [Todos]
*      summary: Buat todo baru
*      requestBody:
*        required: true
*        content:
*          application/json:
*            schema:
*              type: object
*              required: [title]
*              properties:
*                title:
*                  type: string
*      responses:
*        201:
*          description: Todo berhasil dibuat
*          content:
*            application/json:
*              schema:
*                $ref: '#/components/schemas/Todo'
*        400:
*          description: Title wajib diisi
*/
router.post('/todos', (req: Request, res: Response) => {
  const { title } = req.body;
  if (!title) {
    return res.status(400).json({ message: 'Title wajib diisi' });
  }
  const newTodo: Todo = {
    id: todos.length + 1,
    title,
    completed: false,
  };
  todos.push(newTodo);
  res.status(201).json(newTodo);
});

export default router;

```

3. Schema definitions

```

// src/docs/schemas.ts
/**
 * @openapi
 * components:
 *   schemas:

```

```

*      Todo:
*      type: object
*      properties:
*        id:
*          type: integer
*          example: 1
*        title:
*          type: string
*          example: "Belajar REST API"
*        completed:
*          type: boolean
*          example: false
*      Error:
*      type: object
*      properties:
*        success:
*          type: boolean
*          example: false
*        message:
*          type: string
*          example: "Resource not found"
*/
export {};

```

4. Mount Swagger UI

```

// src/app.ts
import express from 'express';
import swaggerUi from 'swagger-ui-express';
import { swaggerSpec } from './docs/swagger';
import todoRoutes from './routes/todos';

const app = express();
app.use(express.json());

// Swagger UI
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec, {
  customCss: '.swagger-ui .topbar { display: none }',
  customSiteTitle: 'Todo API Docs',
}));

// JSON version spec (buat consume programmatically)
app.get('/api-docs.json', (_req, res) => {
  res.json(swaggerSpec);
});

```

```
app.use('/api/v1', todoRoutes);

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server jalan di http://localhost:${PORT}`);
  console.log(`Docs: http://localhost:${PORT}/api-docs`);
});
```

5. Hasil

Buka <http://localhost:3000/api-docs> — Swagger UI siap dipake. Bisa “Try it out” langsung dari browser.

Latihan

1. Dari route GET `/api/products` dan POST `/api/products`, tulis JSDoc OpenAPI annotation lengkap (parameter query untuk filter, request body, responses 200/201/400).
2. Tambahin schema Product di `components/schemas` dengan properti: `id` (integer), `name` (string), `price` (number), `category` (string), `inStock` (boolean). Pake JSDoc comment.
3. Setup swagger-jsdoc + swagger-ui-express di Express project. Include custom CSS buat hide topbar. Mount di `/docs`. Tulis kode setup lengkap (TypeScript).
4. Bikin endpoint GET `/api/users/:id` dengan JSDoc OpenAPI. Parameter path `id`, response 200 (User ditemukan), 404 (User not found). Include example values di schema.

28.3 Error Handling & Pagination

Standard Error Format — Problem Details (RFC 7807)

Daripada setiap endpoint return format error beda-beda, pake standar **RFC 7807**:

```
// Struktur RFC 7807 Problem Details
interface ProblemDetails {
  type: string;           // URI yang ngejelasin error type
  title: string;          // Judul singkat (human-readable)
  status: number;         // HTTP status code
  detail: string;         // Penjelasan detail
```

```

instance?: string; // URI spesifik yang error (optional)
// Bisa tambah field custom
errors?: Record<string, string[]>; // field-level errors
}

```

Contoh Response

```

// 422 Validation Error
{
  "type": "https://api.example.com/errors/validation-error",
  "title": "Validation Error",
  "status": 422,
  "detail": "Data yang dikirim gak valid",
  "instance": "/api/v1/users",
  "errors": {
    "email": ["Email udah terdaftar", "Format email salah"],
    "name": ["Nama minimal 3 karakter"]
  }
}

// 404 Not Found
{
  "type": "https://api.example.com/errors/not-found",
  "title": "Resource Not Found",
  "status": 404,
  "detail": "User dengan ID 99 gak ditemukan",
  "instance": "/api/v1/users/99"
}

// 429 Too Many Requests
{
  "type": "https://api.example.com/errors/rate-limited",
  "title": "Rate Limit Exceeded",
  "status": 429,
  "detail": "Terlalu banyak request. Coba lagi dalam 60 detik",
  "instance": "/api/v1/todos"
}

```

Implementasi di Express

```

// src/utls/errors.ts
export interface ProblemDetails {
  type: string;
  title: string;
  status: number;
}

```

```

    detail: string;
    instance?: string;
    errors?: Record<string, string[]>;
  }

  export class AppError extends Error {
    public readonly status: number;
    public readonly type: string;
    public readonly errors?: Record<string, string[]>;

    constructor(status: number, title: string, detail: string, errors?: Record<string, string[]>) {
      super(detail);
      this.status = status;
      this.type = `https://api.example.com/errors/${title.toLowerCase().replace(/ /g, '-')}`;
      this.title = title;
      this.errors = errors;
    }
  }

  export function sendError(res: any, error: AppError, instance?: string): void {
    const body: ProblemDetails = {
      type: error.type,
      title: error.title,
      status: error.status,
      detail: error.message,
      instance: instance || res.req?.originalUrl,
    };
    if (error.errors) {
      body.errors = error.errors;
    }
    res.status(error.status).json(body);
  }

```

Error Middleware Global

```

// src/middleware/errorHandler.ts
import { Request, Response, NextFunction } from 'express';
import { AppError, sendError } from '../utils/errors';

export function errorHandler(err: Error, req: Request, res: Response, _next: NextFunction) {
  if (err instanceof AppError) {
    return sendError(res, err, req.originalUrl);
  }

  // Unknown error – jangan bocorin detail

```

```

    console.error('UNEXPECTED ERROR:', err);
    const internalError = new AppError(500, 'Internal Server Error', 'Terjadi error');
    sendError(res, internalError, req.originalUrl);
  }

  // Di app.ts
  app.use(errorHandler);

```

Error Codes & Messages — Best Practice

Bikin enum/constant biar konsisten:

```

// src/constants/errors.ts
export const ErrorCodes = {
  VALIDATION_ERROR: { status: 422, title: 'Validation Error' },
  NOT_FOUND: { status: 404, title: 'Resource Not Found' },
  DUPLICATE_ENTRY: { status: 409, title: 'Duplicate Entry' },
  UNAUTHORIZED: { status: 401, title: 'Unauthorized' },
  FORBIDDEN: { status: 403, title: 'Forbidden' },
  RATE_LIMITED: { status: 429, title: 'Rate Limit Exceeded' },
  INTERNAL: { status: 500, title: 'Internal Server Error' },
} as const;

```

Pagination

Offset Pagination (paling umum)

GET /api/users?page=2&limit=10

- page = halaman ke berapa (mulai 1)
- limit = jumlah item per halaman
- offset = (page - 1) * limit

```

// Express pagination handler
app.get('/api/users', (req: Request, res: Response) => {
  const page = Math.max(1, parseInt(req.query.page as string) || 1);
  const limit = Math.min(100, Math.max(1, parseInt(req.query.limit as string) || 1));
  const offset = (page - 1) * limit;

  const paginatedUsers = users.slice(offset, offset + limit);
  const total = users.length;
  const totalPages = Math.ceil(total / limit);

  res.json({
    success: true,
    data: paginatedUsers,
    pagination: {

```

```

        page,
        limit,
        total,
        totalPages,
        hasNext: page < totalPages,
        hasPrev: page > 1,
      },
    ));
  });
});

```

Response:

```

{
  "success": true,
  "data": [...],
  "pagination": {
    "page": 2,
    "limit": 10,
    "total": 47,
    "totalPages": 5,
    "hasNext": true,
    "hasPrev": true
  }
}

```

Cursor Pagination (buat real-time / infinite scroll)

GET /api/users?cursor=eyJpZCI6MTB9&limit=10

- cursor = token base64/encoded yang nunjukin posisi terakhir
- Biasanya pake id atau createdAt sebagai cursor

```

interface CursorPaginationParams {
  cursor?: string;
  limit: number;
}

```

```

function decodeCursor(cursor: string): { id: number } | null {
  try {
    const decoded = Buffer.from(cursor, 'base64').toString('utf-8');
    return JSON.parse(decoded);
  } catch {
    return null;
  }
}

```

```

app.get('/api/users-cursor', (req: Request, res: Response) => {

```

```

const limit = Math.min(100, parseInt(req.query.limit as string) || 10);
const cursorParam = req.query.cursor as string | undefined;

let cursorId = 0;
if (cursorParam) {
  const decoded = decodeCursor(cursorParam);
  if (!decoded) {
    return res.status(400).json({ message: 'Cursor tidak valid' });
  }
  cursorId = decoded.id;
}

const filteredUsers = users.filter(u => u.id > cursorId);
const paginatedUsers = filteredUsers.slice(0, limit);
const nextCursor = paginatedUsers.length === limit
  ? Buffer.from(JSON.stringify({ id: paginatedUsers[paginatedUsers.length - 1]
    : null;

res.json({
  success: true,
  data: paginatedUsers,
  pagination: {
    limit,
    hasMore: nextCursor !== null,
    nextCursor,
  },
});
});

```

Offset vs Cursor — Kapan Pake

Aspek	Offset	Cursor
Use case	Table biasa, halaman manual	Infinite scroll, real-time feed
Performance	Makin besar offset makin lambat (OFFSET scan)	Stabil (WHERE id > X)
Data berubah	Item baru bikin halaman geser (duplicate/skip)	Gak terpengaruh insert baru
User experience	Bisa lompat ke halaman tertentu	Gak bisa lompat ke halaman acak
Implementasi	Sederhana	Agak kompleks

Sorting & Filtering

```
// GET /api/users?sort=name&order=asc&role=admin&status=active
app.get('/api/users', (req: Request, res: Response) => {
  let result = [...users];

  // Filtering
  if (req.query.role) {
    result = result.filter(u => u.role === req.query.role);
  }
  if (req.query.status) {
    result = result.filter(u => u.status === req.query.status);
  }

  // Sorting
  const sortField = (req.query.sort as string) || 'id';
  const sortOrder = (req.query.order as string) === 'desc' ? -1 : 1;

  result.sort((a: any, b: any) => {
    if (a[sortField] < b[sortField]) return -1 * sortOrder;
    if (a[sortField] > b[sortField]) return 1 * sortOrder;
    return 0;
  });

  // Pagination
  const page = parseInt(req.query.page as string) || 1;
  const limit = parseInt(req.query.limit as string) || 10;
  const offset = (page - 1) * limit;
  const paginated = result.slice(offset, offset + limit);

  res.json({
    success: true,
    data: paginated,
    pagination: {
      page, limit,
      total: result.length,
      totalPages: Math.ceil(result.length / limit),
    },
  });
});
```

Rate Limiting — Headers

express-rate-limit otomatis nambahin headers:

RateLimit-Limit: 100

```

RateLimit-Remaining: 87
RateLimit-Reset: 1712345678
Retry-After: 45

import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 60 * 1000, // 1 menit
  max: 100,             // max 100 request per menit
  standardHeaders: true, // Return RateLimit-* headers
  legacyHeaders: false, // Gak pake X-RateLimit-* (old)
  message: {
    type: 'https://api.example.com/errors/rate-limited',
    title: 'Rate Limit Exceeded',
    status: 429,
    detail: 'Terlalu banyak request. Coba lagi.',
  },
});

app.use('/api/', limiter);

```

Latihan

1. Implement `AppError` class dan global error handler middleware di Express TypeScript. Include `sendError` helper. Test dengan `throw AppError(404, 'Not Found', 'User tidak ditemukan')`.
2. Bikin endpoint `GET /api/products` dengan offset pagination (page, limit) + sorting (sort, order) + filtering (minPrice, maxPrice, category). Return pagination metadata.
3. Ubah pagination di endpoint `/api/products` jadi cursor-based. Pake field `id` sebagai cursor. Return `nextCursor` kalo masih ada data.
4. Tambahin `express-rate-limit` middleware dengan custom message RFC 7807. Konfigurasi: 50 request per 15 menit per IP. Return `ProblemDetails` kalo kena limit.

28.4 Versioning & Security

API Versioning

Kenapa perlu versioning? Biar perubahan API gak ngerusak client lama.

1. URL Path Versioning (paling umum)

GET /api/v1/users

GET /api/v2/users

```
// src/routes/v1/users.ts
import { Router } from 'express';
const router = Router();

router.get('/users', (_req, res) => {
  res.json({ version: 'v1', users: [] });
});

export default router;

// src/routes/v2/users.ts
const routerV2 = Router();

routerV2.get('/users', (_req, res) => {
  res.json({
    version: 'v2',
    data: { users: [] },
    meta: { page: 1, total: 0 },
  });
});

export default routerV2;

// src/app.ts
import v1UserRoutes from './routes/v1/users';
import v2UserRoutes from './routes/v2/users';

app.use('/api/v1', v1UserRoutes);
app.use('/api/v2', v2UserRoutes);
```

Pro: Gampang, jelas, bisa coexist.

Kontra: URL berubah tiap versi.

2. Header Versioning

GET /api/users

Accept: application/vnd.myapp.v1+json

```
app.get('/api/users', (req: Request, res: Response) => {
  const accept = req.headers['accept'] || '';
  const versionMatch = accept.match(/application\/vnd\.myapp\.v(\d+)\+json/);
  const version = versionMatch ? parseInt(versionMatch[1]) : 1;
```

```

    if (version === 1) {
      return res.json({ users: [] });
    }
    if (version === 2) {
      return res.json({ data: { users: [] }, meta: { page: 1 } });
    }
    res.status(400).json({ message: 'Versi API gak didukung' });
  });

```

3. Query Parameter Versioning

GET /api/users?version=1

GET /api/users?version=2

```

app.get('/api/users', (req: Request, res: Response) => {
  const version = parseInt(req.query.version as string) || 1;

  if (version === 1) return res.json({ users: [] });
  if (version === 2) return res.json({ data: { users: [] } });
  res.status(400).json({ message: 'Versi gak didukung' });
});

```

Tabel Perbandingan

Strategy	Kelebihan	Kekurangan
URL Path	Paling jelas, mudah di-cache, bisa deploy bareng	URL berubah, routing agak repetitive
Header	URL bersih, gak perlu routing terpisah	Client ribet set header, susah di-cache
Query	Gampang di-test di browser	URL jadi panjang, polusi query params

Rekomendasi: Pake URL path versioning buat public API.
Pake header versioning buat internal API.

Backward Compatibility

Aturan backward compatibility:

⚠ Breaking	🟢 Aman
Rename field (name → fullName)	Nambah field baru
Hapus endpoint	Nambah endpoint baru
Ubah tipe data (number → string)	Nambah optional parameter
Hapus required field	Nambah optional field
Ubah status code	Nambah status code baru

Deprecation Strategy

```
// src/middleware/deprecation.ts
import { Request, Response, NextFunction } from 'express';

export function deprecationMiddleware(deprecatedSince: string, sunsetDate?: string) {
  return (req: Request, res: Response, next: NextFunction) => {
    res.setHeader('Deprecation', `true`);
    res.setHeader('Deprecated-Since', deprecatedSince);
    if (sunsetDate) {
      res.setHeader('Sunset', new Date(sunsetDate).toUTCString());
      res.setHeader('Link', `<https://docs.api.example.com/migration>; rel="migration"`);
    }
    next();
  };
}

// Pake di route lama yang mau di-deprecate
app.get('/api/v1/users', deprecationMiddleware('2024-01-01', '2024-06-30'), (req, res) => {
  res.json({ users: [] });
});
```

Response headers:

```
Deprecation: true
Deprecated-Since: 2024-01-01
Sunset: Sun, 30 Jun 2024 00:00:00 GMT
Link: <https://docs.api.example.com/migration>; rel="migration"
```

Security

1. API Keys

```
// src/middleware/apiKey.ts
import { Request, Response, NextFunction } from 'express';

const API_KEYS = new Set(['sk-abc123', 'sk-def456']);
```

```

export function apiKeyMiddleware(req: Request, res: Response, next: NextFunction) {
  const apiKey = req.headers['x-api-key'] as string;

  if (!apiKey) {
    res.status(401).json({
      type: 'https://api.example.com/errors/unauthorized',
      title: 'Unauthorized',
      status: 401,
      detail: 'API key gak ditemukan. Kirim di header X-API-Key',
    });
    return;
  }

  if (!API_KEYS.has(apiKey)) {
    res.status(403).json({
      type: 'https://api.example.com/errors/forbidden',
      title: 'Forbidden',
      status: 403,
      detail: 'API key gak valid',
    });
    return;
  }

  next();
}

```

2. JWT (JSON Web Token)

```

npm install jsonwebtoken bcrypt
npm install -D @types/jsonwebtoken @types/bcrypt

// src/utils/auth.ts
import jwt from 'jsonwebtoken';
import bcrypt from 'bcrypt';

const JWT_SECRET = process.env.JWT_SECRET || 'rahasia-super-aman';

export function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, 12);
}

export function comparePassword(password: string, hash: string): Promise<boolean> {
  return bcrypt.compare(password, hash);
}

export function generateToken(userId: number, role: string): string {

```

```

    return jwt.sign({ userId, role }, JWT_SECRET, { expiresIn: '7d' });
  }

  export function verifyToken(token: string): { userId: number; role: string } {
    return jwt.verify(token, JWT_SECRET) as { userId: number; role: string };
  }

  // src/middleware/auth.ts
  import { Request, Response, NextFunction } from 'express';
  import { verifyToken } from '../utils/auth';

  // Extend Request type
  declare global {
    namespace Express {
      interface Request {
        user?: { userId: number; role: string };
      }
    }
  }

  export function authMiddleware(req: Request, res: Response, next: NextFunction):
    const authHeader = req.headers['authorization'];

    if (!authHeader) {
      res.status(401).json({ message: 'Token gak ditemukan' });
      return;
    }

    const token = authHeader.startsWith('Bearer ')
      ? authHeader.slice(7)
      : authHeader;

    try {
      const payload = verifyToken(token);
      req.user = payload;
      next();
    } catch {
      res.status(401).json({ message: 'Token gak valid atau expired' });
    }
  }

  export function requireRole(...roles: string[]) {
    return (req: Request, res: Response, next: NextFunction): void => {
      if (!req.user || !roles.includes(req.user.role)) {
        res.status(403).json({ message: 'Gak punya akses' });
        return;
      }
    }
  }

```

```

    }
    next();
  };
}

// Pake di route
app.post('/api/v1/auth/login', async (req: Request, res: Response) => {
  const { email, password } = req.body;
  // ... verify user dari database
  const token = generateToken(1, 'admin');
  res.json({ token });
});

app.get('/api/v1/profile', authMiddleware, (req: Request, res: Response) => {
  res.json({ user: req.user });
});

app.delete('/api/v1/users/:id',
  authMiddleware,
  requireRole('admin'),
  (req: Request, res: Response) => {
    // Hanya admin bisa hapus user
    res.json({ message: 'User dihapus' });
  }
);

```

3. CORS

```

npm install cors
npm install -D @types/cors

import cors from 'cors';

// Allow semua origin (development)
app.use(cors());

// Production - strict
app.use(cors({
  origin: ['https://frontend-saya.com', 'https://admin-saya.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-API-Key'],
  exposedHeaders: ['RateLimit-Limit', 'RateLimit-Remaining'],
  credentials: true,
  maxAge: 86400, // Preflight cache 24 jam
}));

```

4. Helmet (Security Headers)

```
npm install helmet
```

```
import helmet from 'helmet';

app.use(helmet());
// Otomatis nambahin headers:
// - X-Content-Type-Options: nosniff
// - X-Frame-Options: SAMEORIGIN
// - X-XSS-Protection: 0
// - Strict-Transport-Security: max-age=15552000; includeSubDomains
// - Content-Security-Policy: ...
```

5. Logging (Morgan + Winston)

```
npm install morgan winston
```

```
npm install -D @types/morgan
```

```
// src/utils/logger.ts
```

```
import winston from 'winston';

export const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss' }),
    winston.format.errors({ stack: true }),
    winston.format.json(),
  ),
  transports: [
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.printf(({ timestamp, level, message, stack }) => {
          return `${timestamp} [${level}]: ${message}${stack ? '\n' + stack : ''}`;
        })
      ),
    }),
    new winston.transports.File({ filename: 'logs/error.log', level: 'error' }),
    new winston.transports.File({ filename: 'logs/combined.log' }),
  ],
});

// src/app.ts
import morgan from 'morgan';
import { logger } from '../utils/logger';
```

```

// Morgan stream ke Winston
const morganStream = {
  write: (message: string) => logger.info(message.trim()),
};

app.use(morgan('combined', { stream: morganStream }));

// Contoh output log:
// 2024-07-04 14:30:00 [info]: ::1 - - "GET /api/v1/users HTTP/1.1" 200 45

```

Full App Setup — Semua Jadi Satu

```

// src/app.ts – full production-ready setup
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import rateLimit from 'express-rate-limit';
import { authMiddleware } from './middleware/auth';
import { logger } from './utils/logger';

const app = express();

// 1. Security headers
app.use(helmet());

// 2. CORS
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') || '*',
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
}));

// 3. Rate limiting
app.use('/api/', rateLimit({
  windowMs: 15 * 60 * 1000, // 15 menit
  max: 100,
}));

// 4. Body parser
app.use(express.json({ limit: '10kb' }));

// 5. Logging
app.use(morgan('combined', { stream: { write: (msg: string) => logger.info(msg.t

// 6. Routes (v1 & v2)

```

```

import v1UserRoutes from './routes/v1/users';
import v2UserRoutes from './routes/v2/users';
app.use('/api/v1', v1UserRoutes);
app.use('/api/v2', authMiddleware, v2UserRoutes);

// 7. Global error handler
app.use((err: Error, _req: express.Request, res: express.Response, _next: express.NextFunction) => {
  logger.error(err.message, { stack: err.stack });
  res.status(500).json({
    type: 'https://api.example.com/errors/internal',
    title: 'Internal Server Error',
    status: 500,
    detail: 'Terjadi error di server',
  });
});

export default app;

```

Latihan

1. Implement URL path versioning (/api/v1/products dan /api/v2/products) dimana v1 return { products: [] } dan v2 return { data: { products: [] }, meta: { page, total } }. Pake Express Router terpisah.
2. Bikin auth middleware (JWT) dengan 3 role: admin, user, guest. Endpoint DELETE /api/v1/users/:id cuma bisa diakses admin. Kalo user role lain — return 403.
3. Setup helmet, cors, dan rate limiter di Express app. Cors cuma allow http://localhost:5173. Rate limit 50 request per 15 menit. Tulis kode setup lengkap.
4. Integrasi morgan (logging request) + winston (logging error ke file). Error log disimpan di logs/error.log. Request log pake format combined. Tulis kode lengkap + middleware.